Universal Review Mining & Intelligence System

Comprehensive Technical Project Documentation

Project type: Client-side business-intelligence and review-analytics dashboard with an optional Gemini-powered executive-summary API.
Primary language: TypeScript. **UI:** React 19, Vite, Tailwind CSS 4. **Server:** Node.js and Express.

1. Purpose and scope

The system accepts customer-review datasets and transforms them into an interactive business-intelligence dashboard. It is intended for product, hotel, restaurant, and service feedback. It supports browser-based ingestion, transparent data preparation, explainable text analytics, interactive filtering, and downloadable reports. Built-in samples cover mobile devices, hospitality, and restaurants.

2. Feature inventory

- CSV, XLSX, and XLS upload with built-in example datasets.
- Automatic review/rating/product/date/category/price detection with editable mapping.
- Data-quality profiling, configurable cleaning, reusable NLP tokens, and sentiment classification.
- Interactive EDA, KPIs, Plotly charts, text mining, topic/aspect analysis, association rules, and automated insights.
- Downloadable cleaned CSV and self-contained HTML intelligence report.
- Optional server-side Gemini executive summary and a Python/Streamlit reference-code exporter.

3. Technology stack

Layer	Technology	Purpose
Frontend	TypeScript, React 19, React DOM	Dashboard state, components, and client-side analytics orchestration.
Build	Vite 6, esbuild, tsx, TypeScript	Development server, web build, server bundling, type checking.
Styling	Tailwind CSS 4 and custom CSS	Responsive layout, tabs, filters, cards, tables, states.
Charts	Plotly.js and react-plotly.js	Interactive business-intelligence visualizations.
File ingestion	Papa Parse and SheetJS xlsx	Browser parsing of CSV, XLSX, and XLS files.
Server	Node.js, Express 4, dotenv	Health route, Gemini proxy, development and production serving.
AI	@google/genai	Optional Gemini executive synthesis.
UI extras	lucide-react, motion	Icons; Motion is installed but not used in inspected source.

4. Architecture and data flow

Most analysis runs entirely in the browser. React holds raw rows, the selected column mapping, cleaning configuration, filters, and derived results in memory. useMemo recalculates downstream analytics whenever data, configuration, or filters change. No database, authentication, persistent user storage, or background job system is implemented.

1. User selects a sample dataset or uploads CSV/XLSX/XLS.
2. The loader parses rows and headers; the detector proposes semantic columns.
3. The user confirms or adjusts mapping; cleaning produces canonical review records and tokens.
4. Dashboard filters create the active subset used by EDA, sentiment, text mining, topics, aspects, rules, and insights.
5. Tabs render typed results; exports create a cleaned CSV or HTML report in the browser.
6. An optional Express endpoint sends approved metrics and sample reviews to Gemini for executive text synthesis.

5. Core data model

Type	Important fields	Role
ColumnMapping	review, rating, product, date, category, price	User-approved dataset interpretation.
CleaningConfig	removal flags, min length, custom stopwords	Text-preprocessing behavior.
CleanedReviewRow	original/clean text, tokens, mapped fields, sentiment	Canonical analysis-ready review record.
DataQualityReport	dimensions, missing/duplicate counts, types	Dataset health profile.
EDAMetrics	distributions, rankings, timeline, scatter data	Dashboard analytics payload.
SentimentAnalysisResult	counts, score, matrix-style metrics	Sentiment output and diagnostic alignment.
TextMiningResult	frequencies, n-grams, TF-IDF-like scores	Text-analysis payload.
TopicInfo / Aspect / Rule	keywords, proportions, examples	Specialized discovery outputs.

6. Analysis engines

File loading

dataLoader.ts uses Papa Parse for headed CSV files and SheetJS for the first XLSX/XLS worksheet. It returns rows, columns, dimensions, metadata, and errors.

Column detection

columnDetector.ts samples up to 50 rows, profiles average length/numeric/date characteristics, matches header keywords, and falls back to the longest non-numeric text column for reviews and a constrained numeric column for ratings.

Data quality

dataQuality.ts counts blank cells, serialized duplicate rows, cardinality, basic inferred types, and approximate JSON-size memory.

Cleaning

dataCleaner.ts removes missing/duplicate reviews, strips HTML/URLs, normalizes text, removes stopwords/numbers/single-character tokens, applies suffix-based lemmatization, parses ratings/prices, and carries mapped fields forward.

Sentiment

sentimentEngine.ts uses curated positive/negative lexicons, preceding-token intensifier/negation rules, and 70% lexical plus 30% star-rating blending. Scores above 0.15 are positive; below -0.15 are negative.

EDA

edaEngine.ts calculates review/rating/product/category totals, distributions, rankings, monthly counts, review-length buckets, and limited scatter series.

Text mining

textMiningEngine.ts creates overall and sentiment-specific word counts, bigrams/trigrams, a smoothed TF-IDF-like score, and color-tagged word-cloud items.

Topics

topicModelingEngine.ts ranks fixed domain keyword clusters, then falls back to grouped high-frequency terms. This is a heuristic, not statistical LDA.

Aspect sentiment

aspectSentimentEngine.ts matches fixed multi-domain aspect vocabularies and aggregates sentiment counts plus representative reviews. A review can match multiple aspects.

Association rules

associationMiningEngine.ts converts reviews to token sets, mines single and pair itemsets, and produces directional support/confidence/lift rules. It returns the top 15 rules by lift.

Insights and exports

insightsEngine.ts creates deterministic factual statements. reportGenerator.ts outputs quote-escaped CSV and self-contained HTML reports.

7. User interface modules

Area	Main source	Role
Application orchestrator	App.tsx	Owns state, mappings, filters, memoized analytics, tab routing, downloads.
Navigation and filters	Navbar.tsx; SidebarFilters.tsx	Sticky navigation and collapsible BI slicers.
Ingestion/preparation	FileUpload, ColumnDetection, DataCleaner sections	File selection, schema review, cleaning setup.
Metrics and charts	KPICardsGrid; EDADashboard; PlotlyChart	KPI cards and interactive analytical views.
Analysis displays	Sentiment, TextMining, Topic, Aspect, Association sections	Renders engine-specific result views.
Tabs and export	components/tabs; TablesDownloads; PythonExporter	Organizes 12 screens, exports, and Python reference content.

8. Dashboard screens

Screen	Capabilities
Dataset & Ingestion	Upload, sample selection, metadata, parse errors, editable column mapping.
Overview / Quality	High-level KPIs, missing/duplicate data, inferred types, unique values, memory estimate.
EDA / Sentiment	Rating/product/category/time/length analysis, sentiment distribution, rating/text mismatch view.
Text / Data Mining	Frequent terms, n-grams, TF-IDF-like values, word clouds, itemsets and association rules.
Topics / Product Comparison	Seeded topic clusters, domain aspects, product comparison and sentiment breakdown.
Insights / Downloads	Deterministic insight cards, cleaned CSV, HTML report, and Python reference code.

9. API, setup, and commands

GET /api/health returns service status. POST /api/gemini/summarize accepts aggregate metrics, sample reviews, and a domain name. It requests key drivers, customer friction, and strategic recommendations from gemini-3.6-flash; it returns { summary }. A missing GEMINI_API_KEY returns HTTP 400, while model/API failures return HTTP 500.

Command	Purpose
npm install	Install JavaScript and TypeScript dependencies.
npm run dev	Run tsx server.ts; Express listens on port 3000 with Vite middleware.
npm run build	Build the Vite client and bundle server.ts to dist/server.cjs.
npm start	Start the production server bundle.
npm run preview	Run Vite static preview.
npm run lint	Run TypeScript type checking using tsc --noEmit.

Optional AI configuration: GEMINI_API_KEY=your_key_here in server environment configuration. The key must never be included in browser source or exposed Vite environment variables.

10. Security and operational notes

- Default ingestion and analysis happen in the browser; device memory limits apply to large files.
- Using Gemini transmits the supplied metrics and sample reviews to Google. Do not send sensitive content without an approved sharing policy.
- The source contains no authentication, authorization, database, persistence, audit log, encrypted storage, rate limiting, or API validation.
- A production deployment should add HTTPS, validation, rate limits, observability, managed secrets, and a privacy/retention policy.

11. Current limitations

- Sentiment is explainable but lexicon-based, not a validated trained ML classifier; sarcasm and domain terms may be misclassified.
- The browser topic method is seeded keyword clustering, not LDA, despite some UI/reference-code wording.
- Association mining handles one- and two-token itemsets, not higher-order Apriori/FP-Growth rules.
- Topic/aspect sidebar selections are represented in UI state but do not currently restrict the active rows. The minimum-price control is also not applied; maximum price is applied.
- Date filters compare strings, so `YYYY-MM-DD` input is safest. Excel loading uses only the first worksheet.
- No dataset/settings persist after refresh. The Python exporter is reference content, not the active runtime.
- Some KPI cards have illustrative fallback values when data is absent; review these before formal reporting.

12. Recommended roadmap

1. Add input/schema validation, worksheet selection, parser diagnostics, and automated unit/end-to-end tests.
2. Apply minimum-price, topic, and aspect filters consistently in the central filtering predicate.
3. Introduce configurable multilingual/domain NLP models and validate them against labeled review datasets.
4. Add persistence, accounts, access control, export history, audit logging, and production observability.
5. Improve accessibility for charts and controls, including keyboard support and non-visual data alternatives.

13. Summary

This project is a TypeScript/React review-intelligence dashboard with browser-side ingestion, configurable cleaning, explainable NLP heuristics, exploratory analytics, association rules, aspect/topic summaries, and exportable business reports. Express supplies a small API layer and optional Gemini synthesis. Its next maturity step is stronger validation, complete filter semantics, persistent/security-aware operations, and evaluated NLP methods.